1st chapter

1. first palindrome string

```
words = ["abc", "car", "ada", "racecar",
 "cool"]
   ans = " "
for word in words:
  reverse = word [::-1]
  if word == reverse:
    ans = word
  break
print ("First palindrome:", answer)
```

I/p :-
["abc", "car", "ada", "racecar", "cool"]

O/p :- first palindrome : ada

2. Coont : common Elements

```
nums1 = [2,3,2)
nums 2 = [1,2]

ans1 = 0
ans2 = 0,
for i in nums1:
 if i in nums2:
    answer 1 += 1
for i in nums 2:
 if i in nums 1:
    answer 2 += 1
answer = [answer 1, ans 2]
   print("Answer:", answer)
```

I/p : nums1 = [2,3,2]
nums 2 = [1,2)
O/p: [2,1]

3. Sum of Squares of distinct counts

```
nums = [1,2,1]
ans = 0
for i in range(len(nums)):
    S = set()
for j in range(i, len(nums)):
    S.add(nums[j])
    ans += len(S) ** 2
    print(answer)
```

O/P:- 15

4. Count valid pairs

```
nums = [3,1,2,2,2,1,3)
k = 2
    Count = 0
for i in range(len(nums)):
for j in range(i+1, len(nums)):
 if nums[i] == nums[j] and (i*j) % k == 0:
    Count += 1
 print(Count)
```

I/P :- nums = [3,1,2,2,2,1,3]
k = 2
O/P : 4

5. Max Element

```
nums = [1,2,3,4,5]
    max = nums[0]
for i in range(1, len(nums)):
if nums[i] > maximum:
```

```
    max = nums[i]
    print [max]
I/p : [1,2,3,4,5]
  O/p : 5
```

6 Sort and final max

```
  nums = [5,2,8,1,9,3)
  if lens (nums)==0 ;
print ("List is empty")
  else:
   nums.sort ()
  print ("Maximum:", nums[-1]):
```

I/p : [5,2,8,1,9,3]

O/p : Max : 9

7. Unique Elements

```
  nums = [3, 7,3,5,2,5,9,2]
    unique = []
  for x in nums:
   if x is not unique:
   unique.append(x)
   print (unique)
```

I/p :- [3,7,9,5,2,5,2,9]

O/p :- [3, 7,5,2,9]

8. Bubble Sort

```
    nums = [5,3,8,4,2]
    n = len (nums)
```

```
for i in range(n):
for j in range(0,n-i-1):
 if nums[j]>nums[j+1]:
nums[j]
Print(nums)
```

I/P [5,3,8,4,2)

O/P [2,3,4,5,8)

9 Binary search

```
while l<=r:
  m=(l+r)//2
if a[m]==x:
print (found at",m)
  break
if a[m]<x:
  l=m+1
  else:
   r=m-1
  else:
  print(" Not found")
```

output:5

10. Merge Sort

```
def sort(a):
   if len(a)<=1:
      return a
   m=len(a)//2
```

```
l = sort(a[:m])
a = [5,2,8,1,3]
print(sort(a))
```

o/p :- [1,2,3,5,8].

11. Ball out of Grid

```
def f(m,N,i,j):
if i<0 or i>=m or j<0 or j>=n:
  return 1
if N==0:
  return 0
print f(2,2,2,0,0)
```

o/p : 6

12 House Robber II

```
def rob(a)
x=y=0
for n in a:
  x,y = y,max(y,x+n)
  return y
a = [2,3,2]
print(max(rob(a[:-1]),rob(a[1:])))
```

o/p: 3

13. climbing stairs

```
n=4
a=b=1
for i in range(n):
    a,b = b, a+b
```

```
print(a)
```

O/P: 5

14 Unique paths

```
m, n = 7, 3
a = [1]*n
for i in range(m-1):
    for j in range(1,n):
        a[j] += a[j-1]
print(a[-1])
```

O/P: 28

15. Large groups

```
while i<len(s):
    j = 1
    while j<len(s) and s[i] == s[j]:
        j += 1
    if j-i >= 3:
        ans.append([i, j-1])
    i = j
print(ans)
```

O/p [3,6]

Selection sort

1.
```
def selection sort (a):
for i in range( len (a)):
    m = i
for j in range (i+1, len(a)):
    if a[j] < a[m]
        m = j
a[i], a[m] = a[m], a[i]
    return a.
a = eval (input (" Enter list:"))
print ("output:" , Selection - Sort(a))
```

I/P: [5, 2, 9, 1, 5, 6]

O/P: [1, 2, 5, 5, 6, 9]

2. Selection Sort

```
a = [5, 2, 9, 1, 5, 6]
  for i in range ( len (a)):
    m = i
  for j in range (i+1, len(a)):
   if a [j] < a [m]:
    m = j
a[i], a[m] = a[m], a[i]
    print (a)
```

I/P: - [5, 2, 9, 1, 5, 6]

O/P [1, 2, 5, 5, 6, 9]

3. Optimized Bubble Sort

```
def bubble_sort(a):
    for i in range(len(a)):
        swapped = False
        for j in range(len(a)-i-1):
            if a[j] > a[j+1]:
                swapped = True
        if not swapped:
            break
    return a
```

I/p : [64, 25, 12, 22, 11]

O/p [11, 12, 22, 25, 64]

4. Insertion Sort with duplicates

```
while j>=0 and a[j]>key:
    a[j+1] = a[j]
    j -= 1
a[j+1] = key
return a
```

I/p: [5, 5, 5, 5]

O/p: [5, 5, 5, 5]

5. k^{th} missing +ve no

```
while k:
    if n not in arr:
        k-=1
    if k==0:
        return n
    n+=1
```

I/P : [2, 3, 4, 7, 11]
5

O/P : 9

6. Peak element

```
while l<r:
    m = (l+r)//2
    if a[m] < a[m+1]:
        l = m+1
    else:
        r = m
    return l
```

I/P :- [1, 2, 3, 1]

O/P :- 2

7. First occurence of string

```
if haystack [i:i+len(needle)]
        == needle:
    return i
return -1
h = input ("Haystack:")
n = input ("Needle:")
```

I/P :- sadbutsad
sad

O/P : 0

8. Substrings of another word

```
for i in range (len (words)):
    for j in range (len(words)):
```

```
        ans.append(words[i])
            break
        return ans
Ip:- ["mass", "as", "hero", "superhero"]
op:- ['as', 'hero']
```

9. closest pair of points

```
for i in range(len(p)):
    for j in range(i+1, len(p)):
        if d < best:
            best, pair = d(P[i], P[j])
```

```
o/p:-
closest pair: (1, 2), (3, 1)
min dt:- 2.236067
```

10 Brute force

```
def hull(P):
    h = []
    for a in p:
    for b in p
        if a == b: continue
    if all cross(a, b, c) >= 0 for
c in p if c not in (a, b)):
            h += [a, b]
        return list
```

o/p:-

[(5,3), (12.5, 7), (15,3), (10,0)
(6,6.5))

11. Convex hull

```
from scipy.spatial import ConvexHull
P = [(1,1) [4,6), (8,1) (0,0) (3,3)]
    h = ConvexHull(P)
Print [P[i] for in h.vertices]
```

o/p:-

(0,0), (8,1), (4,6).

12. TSP - Exhaustive search

```
for x in permutation(C[1:1]):
    route = (start,) + x + (start,)
d = sum (math.dist (route[i], route
        (i+1) for i in range (len(route)-1))
    if d < best:
        best, path = d route.
    return best.path
```

o/p:- Shortest distance : 15.0

13. Assignment problem

```
for p in perm (range (len
(cost))):
```

```
total = sum (cost [i] P[i] for i in
    range (len (cost))
      if total < best:
return best, ans = path:
```

O/P Assignment : (2, 1, 0)

Total cost : 16

14. Knap sack problem

```
for r in range (len(w)+1):
  for x in combination (range(len
                (w)), r)
if (sum w[i] for i in x)
    if value > best:
    best, ans = value, x
    return ans, best
```

O/P

Selection : (0, 2)

Total value : 7

1. Max & Min

```
n = int(input("enter no of elements:"))
a = list(map(int, input().split()))
min = a[0]
max = a[0]
for i in range(1,n):
    if a[i] < min:
        min = a[i]
    if a[i] > max:
        max = a[i]
```

I/P: 8
5 7 3 9 4 12 6 2

O/P: Min = 2
Max = 12

2. Min & Max (Sorted)

```
n = int(input("Enter no of elements:"))
a = list(map(int, input().split()))
print("Min =", a[0])
print("Max =", a[n-1])
```

I/P: 8
2 4 6 8 10, 12, 14, 18

O/P: Min = 2
Max = 18

3. Merge Sort

```
mid = len(a)//2
left = merge_sort(a[:mid])
right = merge_sort(a[mid:])
result = []
i=j=0
while i<len(left) and j<len(right):
    if left[i]>right[j]:
else
    result.append(right)
return result
```

I/p 31 23 35 27 11 21 15 28

O/p [11, 15, 21, 23, 27, 28, 35]

4 Merge Sort with comparisons

```
if len(a)//2
left = merge_sort(a[:mid])
right = merge_sort(a[mid:])
result = []
i=j=0
result += left[i:]
result += left[j:]
```

o/p: Sorted : [1, 4, 12, 23, 25, 67, 78, 87]
Comparisions : 15

5. Quick Sort -

```
pivot = a[0]
left = []
right = []
for x in a[1:]:
    if x <= pivot
        left.append(x)
    else
        right.append(x)
return quicksort(left) + [pivot]
        + quick_sort(right)
```

I/p :- 10 16 8 12 15 63 95
o/p :- [3, 5, 6, 8, 9, 10, 12, 15, 16]

6. Quick Sort - Middle element

```
left = [x for x in a if x<pivot]
middle = [x for x in a if x == pivot]
return quicksort(left) + middle.
```

I/p :- 19 72 35 46 91 22 31
o/p :- 19 22 21 35 46 72 91,

7. Binary Search

```
while low <=high
 mid =(low+high)//2
 if a[mid ]==key.
    break
 elif a(mid)<key
 low=mid +1
 else
  high =mid -1
else
printf ("Element not found")
```

I/P 5 10 15 20 25 30
20
O/P 3

8 Binary search with Mid point

```
if a (mid) = key:
    print (Element found at
                index", mid
    break
 elif a (mid')<key:
    low mid+1
 else
    high =mid-1
```

I/P: 3 9 14 19 25 31 42 47 53

O/P: Mid element : 25

found at : 5

9. K closest points of origin

Points = [1,3] [-2,2], [5,8], [0,1]

K = 2

```
points .sort (key = lambda P: P[0]**2
    + P[1] **2)
print("closest points:")
for i in range(K):
    print (points [i])
```

O/P

closest points

[0, 1]

[-2, 2]

10. Count Tuples

```
for a in A:
for b in B:
 for c in C:
  for d in D
    if a+b+c+d == 0
        Count ++ =1
```

Output

No of tuples: 2

11. Median of Medians

arr = list (map (int, input().split

k = int (input ("Enter k:"))

result = kth_smallest (arr, k)

print ("kth smallest element:", result)

I/P:- 12, 3 5 7 19

O/P: 5

12. Median of Medians

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

K=6

result = median of median (arr, k)

Print kth smallest element:" result

O/P

kth smallest element: 6

13. Meet in Middle.

```
for r in range(len(arr)+1):
    for subset in combinations
        total = sum(subset)
        diff = abs(target - total)
        if diff < best_diff:
            best = subset
```

O/P :- closest subset (3, 1, 3) 2)

sum = 41

14 Exact Sum.

```
def median(a,k)
for r in range(len(a)+1):
    for x in combination(a,r):
        if(sum(x)==target:
            print(x)
            exit(x)
```

O/p :- (1,3,2,9)

15. Strassen Matrix rewrite Multiplication

```
for i in range(2):
    for j in range(2):
```

```
for k in range(2):
C[i][j]+= A[i][k] * B[k][j]
Print(C)
```

O/P [[34, 22], [38, 34]]

```
def Strassen(A,B):
    a = A[0][0]
    b = A[0][1]
    c = A[1][0]
    d = A[1][1]

    e = B[0][0]
    f = B[0][1]
    g = B[1][0]
    h = B[1][1]

    m1 = (a+d) * (e+h)
    m2 = (c+d) * e
    m3 = a * (f-h)
    m4 = d * (g-e)
    m5 = (a+b) * h
    m6 = (c-a) * (e+f)

    C11 = m1+m4-m5+m7
    C12 = m3+m5
    C21 = m2+m4
    C22 = m1-m2+m3+m6
    return [(C11, C12), (C21, C22)]
```

O/P:- [[34, 22], [38, 24]]

4 Unit

1. Dice Throw - Dynamic programming

```
def dice_ways(dice, sides, target):
dp = [[0] *(target+1) for _ in range
                (dice +1)]
    dp[0][0]=1
for d in range (1,dice+1):
    for s in range(1, target+1):
        if s>=face:
    dp[d][s] += dp[d-1][s-face]
return dp [dice] [target]
```

O/P 6

27

2. Assembly line Scheduling

```
a1 = [4,5,3,2]
a2 = [2,10,1,4]

t1 = [7,4,5]
t2 = [9,2,8]

e1, e2 = 10,12
x1, x2 = 18,7

f1 = [e1+a1[0]]
f2 = [e2+a2[0])

for i in range (1,4):
ans = min(f1[3] +x1, f2 [3]+x2)
Print ("Minimum time:", answer)
```

3. Three Assembly Lines

```
a = [
    [5, 9, 3],
    [6, 8, 4],
    [7, 6, 5],
]
t = [
    [0, 2, 3],
    [2, 0, 4],
    [3, 4, 0],
]
dp = [a[0][0], a[1][0], a[2][0]
for i in range (1,3):
    new = []
    for j in range (3):
        best = min (dp[k] + t[k][j] for k
               in range (3))
Print ("Minimum time:", min(dp))
```

4. Minimum Path distance

```
a = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0],
]
n = len(a)
```

```
for k in range (n):
 for i in range(n):
for j in range(n):
a[i][j] = min(a[i][j], a[i][k]+a[k][j])
```

5. Travelling Salu person

```
def dist(a,b):
   if (a,b) in d:
 return d [(a,b)]
 return d [(b,a)]:
 best = None
best_cost = float('inf')
for p in permutations (cities[1:]):
    route = ['A']+ list (P) + ['A']
    cost = sum (dist(route[i], route [i+1])
        if cost < best_cost:
            best = route
```

6. Longest palindromic substring

```
for i in range(len(s)):
for j in range (i, len(s)):
    x = s[i:j+1]
if x == x[::-1] and  len(x) >
        len (longest):
    longest = x
print ("longest palindrome:", longest)
```

7. longest Substring without Repeating characters

```
for ch in s:
    if ch in seen:
        seen = seen[seen.index(ch)+1]
    seen += ch
    if len(seen) > len(longest):
        longest = seen
print("length", len(longest))
```

I/P
abcabcbb
O/P
length: 3

8. word break

```
dp = [False] * (len(s)+1)
dp[0] = ~~False~~ True
for i in range(1, len(s)+1):
    for word in words:
        if s[:i].endswith(word) and
            dp[i-len(word)]:
            dp[i] = True
print(dp[len(s)])
```

O/P
True

9. word break dictionary

```
dp = [False) * (len(s)+1)
```

```
dp[0] = True
for i in range(1, len(s)+1):
    for word in words:
if s[i-len(word) : i] == word and
dp[i-len(word)]:
        dp[i] = True
Print ("yes" if dp[-1] else "No")
```

I/P
ilike samsung

O/P
yes

10. word Filter

```
words = ["apple"]
pref = "a"
Suff = "e"
ans = -1
 for i in range (len(words)):
If words[i].startswith(pref) and
words[i].endswith(stuff):
 Print (ans)
```

O/P 0

11. Floyd's Algo

```
for k in range(n):
    for j in range(n):
  graph[i][j] = min(graph[i][j]
                    graph[i][k]+graph[k]
Print("After:")
for row in graph:
    print(row)
```

O/P

```
[ 0, 3, 999, 999 ]
[ 3, 0, 1, 4 ]
[ 999, 1, 0, 1 ]
[ 999, 4, 1, 0 ]
```

Before

After

```
[ 0, 3, 4, 5 ]
[ 3, 0, 1, 2 ]
[ 4, 1, 0, 1 ]
[ 5, 2, 1, 0 ]
```

12. Floyd's Algo - Router Link Failure

```
def floyd(g):
    n = len(g)
    d = [row[:] for row in g]
    for k in range(n):
  for i in range(n):
    for j in range(n):
d[i][j] = min(d[i][j], d[i][k]+
              d[k][j])
return d.
```

O/P

Before : 5

After : 10

13. Shortest path

```
dist = { x:INF for x in graph }
dist['C'] = 0
for _ in range (len(graph)):
  for u in graph:
for v, w in graph[u].items():
dist[v] = min(dist[v], dist[u] + w)
```

O/P

C to A = 7

14. optimal BST

```
freq = [0.1, 0.2, 0.4, 0.3]
cost = 1.7
Print ("cost:", cost)
```

O/P

Cost: 1.7

Expand

15. optimal BST

```
keys = [10, 12, 16, 21]
freq = [4, 2, 6, 3]
cost = 26
Print ("cost:", cost)
```

Expand

O/P:
Cost: 26

16. Mouse and cat game

```
graph = [
        [2,5],
        [3],
        [0,4,5],
        [1, 4,5],
        [2,3],
        [0,2,3]
    ]
Print("Result:",0)
```

O/P:
Result: 0

17. Max probability path

```
P1 = 0.5
P2 = 0.5
ans = p1*p2.
Print("Max probability:",ans)
```

O/P

Max probability: 0.25

5-unit

1. Maximum coins

```
Piles = [2, 4, 1, 2, 7, 8]
Piles.sort()
coins = 0
i = len(Piles) - 2
while i >= 0:
coins += Piles[i]
i -= 2
Print (coins)
```

O/P 9

2. Min coins to reach target

```
coins = [1, 4, 9]
target = 19
coins.sort()
miss = 1
count = 0
i = 0
while miss <= target:
if i < len(coins) and coins[i] <= miss:
i += 1
else:
miss += miss
count += 1
Print (count)
```

O/P : 2

3 Min Max working Time

```
jobs = [3,2,3)
K =3
jobs.sort (rev=True)
    workers = [0]*K
for job in jobs:
i= workers.index(min(workers))
    workers[i]+=job
print (max(workers)   O/P 3
```

4 Max Job profit

```
for i in range (1, len(jobs)+1):
    S, e, p = jobs[i-1]
take =p
for j in range (i-1):
    if jobs(j)[1]<=S
dp[i] = max [dp[i-1], take)
print (dp[-1])
```

O/P 120

5. Dijkstra's Algo

```
for i in range(n):
if not visited[i] and(u==-1 or
  dist[i]<dist[u]):
    u=i
for v in range(n):
    if graph[u][v]!=INF
  Print (dist)
```

O/p [0, 7, 3, 9, 5]

6. Dijkstra's Algo - edge list

```
for u,v,w in edges:
    graph[u].append(v,w)
    graph[v].append(u,w)
dist = [float('inf')]*n
dist[source] = 0
pq = [(0, source)]
while pq:
    d,u = heapq.heappop(pq)
    for v,w in graph[u]:
        new = d+w
        if new < dist[v]:
    Print(dist[target])
```

O/p 20

7. Huffman coding

```
def make_code(node, code = ""):
    if isinstance(node[1], str):
        codes[nodes[1]] = code
    else:
        make_code(node[0], code+"0")
        make_code(node[1], code+"1")
```

[('c', '0'), ('b', '10'), ('a', '110')]

8. Huffman Decoding

```
for bit in encoded:
    temp += bit
if temp in reverse:
    reselt += rev(temp)
temp = ""
print(result)
```

O/P

abacd

9. Maximom weight in Container

```
weights = [10,20, 30, 40,50]
capacity = 60
weights.sort(rev = True)
total = 0
for w in weights:
    if total + w <= capacity:
        total += w
print (total)
```

O/P: 50

10. Min No of Containers

```
weights = [5,10, 15, 20, 25, 30, 35]
capacity = 50
```

```
for w in weights:
    if current + w <= capacity:
    current += w
else:
  Containers += 1
        current = w
  Print(Containers)
```

O/P 4

11. Kruskali Algo

```
def find(x):
    for u,v,w in edges:
    a = find(u)
     b = find(v)
     if a != b:
     Parent[a] = b
      mst.append(u,v,w)
     total += w
Print("Edges in MST:", mst)
```

O/P

Total wgt: 19

12. Check whether MST is unique

```
weights = [e[2] for e in edges
  if len(weights) == len(set(weights)):
    print("Is the given MST unique")
else
  print("Is the given MST False)
```

O/P

Another possible MST: [(0,1,1), (0,2,1)
(2,3,2), (3,4,3)]

Topic 6 - Back Tracking

1. N Queens

```
def solve(n):
    if r==n: return True
    for c in range(n):
        if all(b[i][c]!="Q" and
(c-(r-i)<0 or b[i][c-(r-i)]!="Q"):
            for i in range(r):
                b[r][c]="Q"
            if bt(r+1): return True
            b[r][c]="."
    return False
```

I/P:- 4

O/P - Q - -
- - - Q
Q - - -
- - Q -

2) 8X8 N-Queens

```
if row==r:
    return True
for col in range(c):
    if all(board[i]!=col and
abs(board[i]-col)!=abs(i-row)
        for i in range(row)
        board[row]=col
```

```
if bt(root1):
    return True
```

Output: [0, 2, 4, 1, 7, 9, 3, 6]

3. Sudoku

```
def solve(b):
    for r in range(9):
        for c in range(9):
            if b[r][c] == 0:
                for n in range(1,10):
                    if n not in b[r] and all
(b[i][c] != n)
range(x, x+3) for j in range
(y, y+3):
    b = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0]
        [6, 0, 0, 1, 9, 5, 0, 0, 0]
        [0, 9, 8, 0, 0, 0, 6, 0]
        [8, 0, 0, 6, 0, 0, 0, 3]
        [4, 0, 8, 0, 3, 0, 0, 1]
            [5, 3, 4, 6, 7, 8, 9, 1, 2]
```

O/P: [6, 7, 2, 1, 9, 5, 3, 4, 5]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]

[4, 2, 6, 8, 5, 3, 7, 9, 1]

5) Target Sum

```
def target(a, t):
    if not a:
        return int(t==0)
    return target(a[1:], t-a[0]) +
        target(a[1:], t+a[0])
a = list(map(int, input().split()))
t = int(input())
print(target(a,t))
```

I/p:- 1 1 1 3 3 o/p: 5

6) Sum of Subarray Minimums

```
for i in range(len(a)):
    m = a[i]
    for j in range(i, len(a)):
        m = min(m, a[j])
        s += m
print(s)
```

I/P: 3 1 2 4 o/p: 17

7) Combination Sum

```
def comb(a, t, i=0, p[]):
    if t == 0:
```

```
        Print(P)
        return
    for j in range(i, len(a)):
      if a[j]<=t:
          comb(a, t-a[j], j, P+(a,j))

comb(a,t)
```

I/P: 2 3 6 7 O/P: [2,2,3]
7 [7]

8) Combination Sum II

```
for r in range(1, len(a)+1):
    for x in combination(sorted(a),r):
        if (sum(x)==t):
            ans.add(x)
for x in ans:
    print(list(x))
```

I/p: 10 1 2 7 6 1 5
8
O/p [1, 1, 6]
[1, 2, 5]
[1, 7]
[2, 6]

9) Permutations

```
from itertools import permutation
a=list(map(int,input))
```

```
for x in permutations(a):
        print(list(x))
```

I/p: 123

o/p: [1,2,3]
[1,3,2]
[2,1,3]
[2,3,1]

10) unique permutation

```
from itertools import permutation
a = list(map(int, input().split()))
for x in sorted(set(permutations
(a))):
    print(list(x))
```

I/p 112 o/p [1,1,2]
[1,2,1]
[2,1,1]

11) Graph coloring

```
for x in range
if all(c[v]!=x for a,b in e
for v in(1,b) if
    c[v]=x
    if b+(v+1):
    return True
```

I/p : 4
3 o/p : [1,2,3,2]

13) Hamiltonian cycle

```
for v in range(1,n):
    if v not in p and g[p[-1]][v]:
        p.append(v)
if bt(): return True
p.pop()
return false
```

I/p :- 5 o/p :- [0,1,2,4,0]

15) All Subsets

```
a = list(map(int, input().split()))
for r in range(len(a)+1):
    for x in combination:
```

I/p 123 o/p
[]
[1]
[2]
[3]
[1,2]
[1,2,3]

16) Subset containing 3

```
for r in range(1, len(a)+1):
    for x in combination(a,r):
        if 3 in x:
            print(list(x))
```

I/p
2345
o/p
[3]
[2,3]
[3,4]

17) Universal

```
a = input().split()
b = input().split()
for x in a:
    if all(not Counter(y) - Counter(x)
```